Torneo de Komikan Concurrente

Sincronización, Competencia en Tiempo Real y Manejo de Recursos

Tarea 2 — Threads, Mutexes, Semáforos y Memoria Compartida

Sistemas Operativos

Fecha de entrega: **30/10/2026** | Modalidad: **Grupos de máximo 2 personas**

1. Descripción

Tras el éxito del Planificador Dieciochero, la comisión organizadora ha decidido llevar la celebración a un nuevo nivel organizando el primer **Torneo Multijugador de Komikan Concurrente en Tiempo Real**.

El **Komikan** es un juego tradicional mapuche asimétrico que enfrenta a un **Puma** contra **12 Perros** en un tablero especial de 5×5 casillas acoplado a un triángulo (trono). A diferencia de los juegos tradicionales de mesa, en este torneo **no existen turnos estrictos**: todos los jugadores compiten y mueven sus fichas de manera simultánea e independiente en milisegundos. La primera ficha que logra reclamar un slot libre del tablero se lo queda.

Para conocer más sobre los orígenes y variantes de las reglas del juego, pueden consultar el siguiente **enlace de referencia sobre el Komikan**.

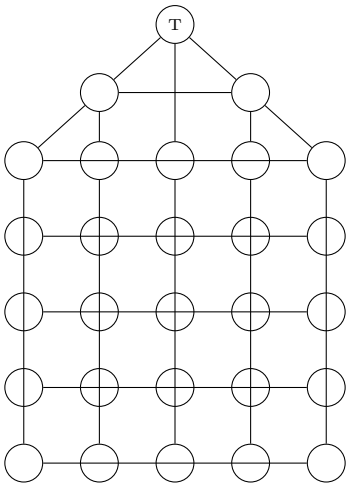
El objetivo de esta tarea es construir la infraestructura completa del torneo: un servidor central de emparejamiento (*Matchmaker*), las mesas de juego concurrentes y los árbitros que gestionan el estado y la sincronización de las partidas.

2. Reglas del Juego y Estructura del Tablero

El estado del tablero debe ser compartido entre las fichas concurrentes de cada partida.

2.1. Estructura del Tablero

El tablero está conformado por una cuadrícula principal de 5×5 intersecciones acoplada en su extremo superior a un triángulo o trono:



2.2. Piezas y Condiciones de Victoria

- **El Puma (1 hilo/proceso):** Se mueve en cualquier dirección a casillas adyacentes libres. Captura a un perro saltando sobre él hacia una casilla adyacente vacía.
  - **Victoria del Puma:** Capturar a 6 o más perros.
- **Los Perros (12 hilos/procesos):** Se mueven a casillas adyacentes libres. Los perros no pueden capturar al puma.
  - **Victoria de los Perros:** Acorralar completamente al Puma, dejándolo sin movimientos válidos posibles.

2.3. Competencia en Tiempo Real (Sin Turnos)

Cada pieza intenta realizar movimientos ejecutando ciclos de decisión con retardos aleatorios en milisegundos (entre 50 y 300 ms). Cuando dos o más fichas intentan ocupar la misma casilla destino de forma simultánea, el sistema de sincronización debe garantizar que solo una adquiera la posición, evitando condiciones de carrera (*race conditions*) y estados inconsistentes.

3. Requisitos Funcionales

Su sistema debe invocarse de la siguiente manera: `./torneo N M [config.txt]` (donde N es el número total de clientes/jugadores a procesar, M es el número máximo de mesas concurrentes y opcionalmente un archivo de configuración).

3.1. Archivo de Configuración de Entrada

Para facilitar una evaluación objetiva, automatizada y escalable en casos borde o con alto número de threads, el sistema debe permitir recibir un archivo de configuración de entrada (`config.txt`). Este archivo definirá parámetros iniciales como:

- Tiempos mínimos y máximos de retardo por movimiento (ms).
- Disposición inicial de fichas en el tablero.
- Semilla aleatoria (*seed*) para reproducibilidad de pruebas.

3.2. Servidor y Emparejamiento (N Jugadores)

- El servidor debe aceptar la llegada de N clientes concurrentes.
- A medida que llegan los jugadores, el servidor debe emparejarlos en grupos de 2 (un jugador Puma y un jugador Perros) y asignarles una **Mesa de Juego** disponible.
- El sistema nunca debe exceder el límite de M mesas activas simultáneamente. Si las M mesas están ocupadas, los pares restantes deben esperar de forma pasiva.

3.3. Juez / Árbitro de Mesa y Simulación del Juego

Cada mesa activa contará con un hilo o proceso **Juez** independiente que:

1. Inicializa el tablero y sus primitivas de sincronización.
2. Valida que los movimientos de las fichas cumplan con las reglas de adyacencia y captura.
3. **Simulación visual/log del juego:** La mesa debe imprimir/simular periódicamente o ante cada movimiento el estado actualizado del tablero (matriz de juego) y los eventos clave en consola o archivo de registro (*log*), permitiendo verificar de forma transparente la competencia en tiempo real.
4. Detecta condiciones de victoria, finaliza la partida y libera la mesa para nuevos jugadores.

4. Resiliencia

4.1. Manejo de Desconexiones / Fallos

Si un jugador aborta o falla internamente durante una partida:

- El Juez debe detectar la anomalía, abortar limpiamente la partida actual declarando al oponente como ganador por abandono.
- Debe desbloquear y destruir cualquier primitiva de sincronización retenida por el proceso fallido.

4.2. Interrupción del Servidor (`Ctrl+C`)

Si el usuario envía una señal `SIGINT` (`Ctrl+C`) al proceso principal:

1. El servidor debe detener inmediatamente la llegada de nuevos jugadores.
2. Debe notificar a todas las partidas activas para que concluyan su ejecución de forma ordenada.
3. Debe liberar **todos** los recursos del sistema utilizados (mutexes, semáforos POSIX/System V, bloques de memoria compartida y descriptores de archivos).
4. El programa debe finalizar garantizando cero fugas de memoria (*memory leaks*) y cero recursos huérfanos en el sistema operativo.

5. Propuesta de Diseño y Análisis

Debido a la naturaleza asimétrica y concurrente del Komikan en tiempo real, **no existe una única arquitectura correcta** para resolver la sincronización del tablero ni la administración de los recursos compartidos.

Cada grupo es libre de diseñar e implementar el esquema de sincronización que considere más conveniente. Sin embargo, junto con el código entregado, se deberá adjuntar un breve informe en PDF (máximo 2 páginas) que contenga:

1. **Diseño del Esquema de Sincronización:** Explicar detalladamente la arquitectura elegida para proteger el tablero y gestionar los recursos compartidos. ¿Cómo garantizaron la exclusión mutua durante los movimientos simultáneos?
2. **Análisis de Riesgos de Concurrencia:** Identificar los potenciales escenarios de *Deadlock*, *Livelock* o *Starvation* que podrían darse en su diseño (por ejemplo, cuando múltiples fichas intentan acceder a las mismas posiciones al mismo tiempo) y explicar la estrategia implementada para prevenirlos o resolverlos.
3. **Evaluación:** Justificar las decisiones de diseño tomadas analizando el balance entre la complejidad de implementación, el rendimiento (*throughput*/latencia de movimientos) y el nivel de concurrencia permitido por su solución.

6. Restricciones Técnicas y Evaluación

- Compilación en C (`gcc -Wall -Wextra -std=c17`) o C++ (`g++ -Wall -Wextra -std=c++17`).
- Uso obligatorio de primitivas POSIX (`pthread_mutex`, `pthread_cond`, `sem_t`) o recursos IPC/System V para la sincronización.
- El código debe compilar y ejecutarse correctamente en entornos Unix. Si la tarea no compila, se evaluará con la nota mínima.
- **Interrogación aleatoria:** Posterior a la entrega se realizará una interrogación individual y aleatoria a los integrantes del grupo para evaluar la autoría y comprensión del diseño concurrente e implementación realizada.